Ruprecht Karl University of Heidelberg
Institute of Computer Science
Artificial Intelligence for Programming

Bachelorarbeit

PandaRec - A Recipe-Based Recommendation System for the Pandas Library

PandaRec - Ein rezeptbasiertes Empfehlungssystem für die Pandas-Bibliothek

Name: Frederick Vandermoeten
Matrikelnummer: 4055264
Betreuer: Prof. Dr. Artur Andrzejak
Datum der Abgabe: 19.10.2023

Hiermit versichere ich, dass ich die Arbeit selbst verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und wörtlich oder inhaltlich aus fremden Werken Übernommenes als fremd kenntlich gemacht habe. Ferner versichere ich, dass die übermittelte elektronische Version in Inhalt und Wortlaut mit der gedruckten Version meiner Arbeit vollständig übereinstimmt. Ich bin einverstanden, dass diese elektronische Fassung universitätsintern anhand einer Plagiatssoftware auf Plagiate überprüft wird.

Heidelberg,

Abstract

Pandas is a Python library that provides data analysis and manipulation tools that are useful for researchers from all fields. However, Pandas' complexity and vast amount of functions can easily overwhelm new users without programming and data science knowledge. In this thesis, I introduce a system that recommends recipes, code snippets with a name and description.

This system uses a graphical user interface (GUI) within Jupyter Notebooks to take advantage of its cell-by-cell execution capabilities. Users can provide a search query and will be given a list of recommended recipes that can directly be used in the Notebook. This should help the user learn the basics of data manipulation with pandas. Additionally, multiple recommendation algorithms will be compared regarding their accuracy and speed. The algorithms will also be compared to current alternatives like ChatGPT.

In conclusion, this thesis aims to empower researchers with a more approachable tool for harnessing the capabilities of Pandas in their work.

Zusammenfassung

Pandas ist eine Python-Bibliothek, die Datenanalyse- und Manipulationswerkzeuge bereitstellt, die für Forscher aus allen Fachbereichen nützlich sind. Die Komplexität von Pandas und die Vielzahl von Funktionen können jedoch neue Benutzer ohne Programmier- und Datenkenntnisse leicht überfordern. In dieser Arbeit stelle ich ein System vor, das Empfehlungen für Rezepte, Code-Schnipsel mit Namen und Beschreibung, bereitstellt.

Dieses System verwendet eine grafische Benutzeroberfläche (GUI) innerhalb von Jupyter Notebooks. Benutzer können eine Suchanfrage eingeben und erhalten eine Liste empfohlener Rezepte, die direkt im Notebook verwendet werden können. Dies soll dem Benutzer helfen, die Grundlagen der Datenmanipulation mit Pandas zu erlernen.

Zusätzlich werden mehrere Empfehlungsalgorithmen hinsichtlich ihrer Genauigkeit und Geschwindigkeit verglichen. Die Algorithmen werden auch mit aktuellen Alternativen wie ChatGPT verglichen. Zusammenfassend zielt diese Arbeit darauf ab, Forschern ein benutzerfreundliches Werkzeug zur Verfügung zu stellen, um die Möglichkeiten von Pandas in ihrer Arbeit effizienter zu nutzen.

Contents

List of Figures 1

List of Tables 2

List of Listings 3

1 Introduction 5

1.1 Motivation . . . 5
1.2 Problems . . . 5
1.3 Solutions . . . 5
1.4 Structure of this Thesis . . . 6

2 Background 7

2.1 Pandas . . . 7
2.2 Natural Language Processing (NLP) . . . 7
2.3 Code Recommendation Systems . . . 7
2.4 Existing Code Recommendation Systems . . . 8
2.4.1 Code Completion . . . 8
2.4.2 Code Generation . . . 8
2.4.3 Generative AI . . . 9
2.5 Searching for Code . . . 9
2.5.1 Lemmatization and Stemming . . . 9
2.5.2 Lexical Search . . . 9
2.5.3 Semantic Search . . . 10
2.5.4 Bidirectional Encoder Representations from Transformers (BERT) 10
2.6 Normalized Discounted Cumulative Gain (NDCG) . . . 11

3 Approach 13

3.1 The Jupyter Notebook GUI . . . 13
3.1.1 The Data Grid and Search Box . . . 13

3.1.2 The Search Results . . . 14
3.1.3 Updating Results . . . 14
3.1.4 Applying the Code Snippet . . . 15
3.2 Recipes . . . 15
3.2.1 Generating Recipes . . . 16
3.2.2 Embeddings . . . 18
3.3 The Recommendation System . . . 18
3.3.1 Context . . . 18
3.3.2 The Recommendation Engine . . . 19
3.3.3 Ranking Strategies . . . 20
3.4 Collecting User Feedback . . . 28

4 Evaluation **29**
4.1 Research Questions . . . 29
4.2 Acceptable Delays . . . 29
4.3 Setup and Search Delays . . . 29
4.3.1 Setup Time of the Solution . . . 30
4.4 Algorithm Accuracy . . . 31
4.4.1 Creating the Ranking Dataset . . . 31
4.4.2 Ranking Based on the Autogenerated Recipes . . . 31
4.4.3 Comparing Semantic Search Models . . . 31
4.4.4 Performance of the Different Recipe Datasets . . . 32
4.4.5 Comparison to OpenAI Chat Completion . . . 33
4.4.6 Comparison to ChatGPT . . . 33
4.4.7 Usefulness of Recipes . . . 33
4.5 Working With Jupyter Notebooks . . . 34

5 Conclusion **35**
5.1 Summary . . . 35
5.2 Future Work . . . 35

Bibliography **37**

List of Figures

2.1 The Levenshtein Distance 10
2.2 Bi vs Cross-Encoder 11

3.1 Recommender Widget 14
3.2 Update Results 15

List of Tables

4.1 Setup and Search Delay 30
4.2 NDCG Scores 31
4.3 NDCG Scores for Different Models 32
4.4 NDCG Scores for Different Datasets 32
4.5 Unachievable Recipes per Dataset 33

List of Listings

3.1 Debounce Recommendations 15
3.2 Recipe Dataclass 16
3.3 Generate Basic Recipes 17
3.4 Context Class 18
3.5 Recommender Class 19
3.6 RankingStrategy Abstract Class 20
3.7 Name Search 21
3.8 Fuzzy Search 22
3.9 Index Search Constructor 23
3.10 Lemmatization 23
3.11 Building the Inverted Index 24
3.12 Searching the Inverted Index 25
3.13 Embedding the Recipes 25
3.14 Searching the Embeddings 26
3.15 OpenAI embeddings 27
3.16 OpenAI System Message 27
3.17 Web Socket Client 28

4.1 Timing index search with a saved index 30

1 Introduction

1.1 Motivation

Pandas is a library for Python that provides data analysis and manipulation tools [12]. While it is powerful and flexible, it is equally complex.

Researchers from all fields often have to work with data. However, researchers or users new to data analysis can struggle to learn how to use Pandas or similar data manipulation software since searching documentation and tutorials is time-intensive. If the user has limited programming knowledge, this problem gets exacerbated since they must learn multiple things simultaneously.

Code recommendation systems are a staple in modern software development. Different methods can help developers write code faster and more efficiently, increase code clarity and maintainability, and help with error detection.

1.2 Problems

A recommendation system for the Pandas library should be fast, easy to set up, easy to use, and useful to all users. That means it should provide the user with relevant recommendations even if they still need to gain domain knowledge. It should also help the user acquire domain knowledge. However, current code recommendation systems only fulfill some of the described requirements.

1.3 Solutions

My solution uses a Jupyter Notebook GUI that anyone can easily set up in the browser with Google Colab or Binder. I will use a set of "recipes" code snippets with at least a name and a description. In the pursuit of developing a sound code recommendation system, we will compare different methods of recommending code and implement them

modularly so that algorithms can be easily exchanged and even composed out of multiple algorithms.

1.4 Structure of this Thesis

In the second chapter, I will present the current state of recommendation systems for Pandas and the techniques that I will use in my recommendation system. The third chapter deals with the design of the GUI and modular recommendation system and then the implementation of the recipes and different algorithms. In the fourth chapter, I will evaluate how the specific ranking algorithms and the different recipe datasets perform compared to each other and other recommendation systems.

2 Background

This chapter will introduce the needed background knowledge to the reader. It will introduce the Pandas library, code recommendation systems, how to search for code, and how to rank our recommendation algorithms.

2.1 Pandas

Pandas is an open-source library for the Python programming language that provides data analysis and manipulation tools. The core of the library are the Series and DataFrame data structures. Series are one-dimensional labeled arrays of any type. DataFrames are two-dimensional data structures that function like a table with rows and columns. Pandas provides tools for data cleaning, manipulation, analysis, and exploration of these data structures. Additionally, tools for data import and export are provided. Pandas also pairs well with visualization libraries like Matplotlib.

2.2 Natural Language Processing (NLP)

Natural language processing is a subfield of artificial intelligence that interprets and generates natural (human) language. In our specific case, we want to query in natural language since this is easier for a new user.

Let us look at existing recommendation systems so we can get an understanding of what our solution should be capable of.

2.3 Code Recommendation Systems

Code recommendation systems help developers write code more efficiently. They can suggest code snippets, highlight errors, and give quick access to documentation. As described by Robillard et al. 2010 [1], code recommendation systems have these design dimensions:

Context is the input of the recommendation system. This can include a large amount of different information. We can look at what the user is typing, the file we are in, the codebase in general, and the interaction history. For Pandas, we can also look at the current DataFrame and its column- and row names, datatypes, and, with a GUI, what part of the DataFrame the user has highlighted.

The Recommendation Engine uses the context and other data sources, such as data from different repositories and bug reports, to rank the recommendation output.

Output can be provided either on demand or continuously and can be multiple suggestions or a single (often inline) suggestion.

User Feedback can change the system in many ways. The user can provide feedback by changing the context and adapting the recommendation engine locally or globally, where feedback either changes the output of the current user or all users.

2.4 Existing Code Recommendation Systems

The code recommendation system we want should be fast, easy to set up, use, and useful to all users. So, let us look at a few existing code recommendation systems and see how they are inadequate for all our goals.

2.4.1 Code Completion

Code completion is already a part of most IDEs. It is a form of autocomplete. In its most usual form, after the user starts typing, it will recommend multiple completions of the typed word. While it helps to reduce typos, it cannot help a new user use a library since basic knowledge of the different functions is required. Code completion makes an experienced user more efficient, but more is needed for inexperienced users. Additionally, it requires an IDE, which will be an additional piece of software that someone who is unfamiliar with computer science would have to learn and set up.

2.4.2 Code Generation

Code recommendation systems can generate entire blocks of code. This can be used for boilerplate code or whole functions. While automated boilerplate can probably help with the basic steps like importing data, some code generation systems already meet many of our requirements.

2.4.3 Generative AI

Generative AI is artificial intelligence that uses large amounts of data to generate similar data [10]. Especially the Transformer [4] has been shown to perform well on natural language processing tasks [5]. Multimodal models like ChatGPT [16] and GitHub Copilot [14], based on OpenAI Codex, are already commonly used to generate code. Using ChatGPT or GitHub Copilot by writing query-like comments performs decently on this task. However, the GPT family requires a large amount of resources to train and run.

2.5 Searching for Code

Since my approach leverages "recipes", a corpus of common data manipulation tasks, I can use search algorithms to search through the descriptions of the recipes. Search algorithms that search for occurrences of the literal query are called lexical search algorithms.

2.5.1 Lemmatization and Stemming

Since words can have different inflections, e.g., walked and walking, they need to be brought to a generic form. Stemming solves this by removing common suffixes. This can, however, result in stems that are not actual words. Lemmatization ensures that the resulting generic form is a valid dictionary entry. It requires linguistic knowledge and a complete vocabulary. Lemmatization is therefore more computationally intensive than stemming but provides more meaningful results.

2.5.2 Lexical Search

In lexical search, the search engine looks for literal matches of the query words [3]. A common and effective way to implement lexical search is with search indexing.

Search Indexing builds an inverted index out of the corpus. First, a forward index is built, which stores word occurrences per document. Then, the inverted index can be constructed, which stores for each word all the annotations that contain it [2].

With this index, the search algorithm can quickly retrieve a list of annotations containing the query.

Fuzzy Search

Fuzzy search uses a string distance function to search for approximately similar strings. The Levenshtein distance is commonly used. In it, the distance amount increases by a constant for every insertion, deletion, and substitution.

$$lev(a,b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ lev(tail(a), tail(b)) & \text{if } a[0] == b[0] \\ \min \begin{cases} lev(tail(a), b) & + \text{ i} \\ lev(a, tail(b)) & + \text{ d} \\ lev(tail(a), tail(b)) & + \text{ c} \end{cases} & \text{else} \end{cases}$$

Figure 2.1: The Levenshtein Distance.

If i + d = c, this reduces to the InDel distance, which only counts insertions and deletions.

2.5.3 Semantic Search

Semantic search can be described as a search for meaning [3]. It aims to understand the meaning of a query instead of literal matches. The Transformer [4] is a deep learning architecture that can capture relationships between words and phrases, which enables context-aware applications. For this reason, Transformer-based models such as the Generative Pre-trained Transformer (GPT) and Bidirectional Encoder Representations from Transformers (BERT) perform well on natural language tasks.

2.5.4 Bidirectional Encoder Representations from Transformers (BERT)

BERT [9] and the more extensively trained RoBERTa [7] are natural language processing models based on the Transformer architecture. They were trained by predicting missing words (masked language model) in sentences from both left and right contexts (bidirectional). This bidirectionality lends itself to sentence pair regression, like sentence similarity. With BERT, both sentences are passed into the model simultaneously, and the similarity is predicted. This is called cross-encoding and produces similarity scores

with great accuracy. However, when cross-encoders compute the similarity between a query and a corpus, they will have to do a cross-encoding for every element of the corpus.

To combat this, Sentence-BERT [8] uses a siamese network structure. Instead of computing the similarity, we compute the sentence embeddings, which can then be compared, often via cosine-similarity or the dot product [17].

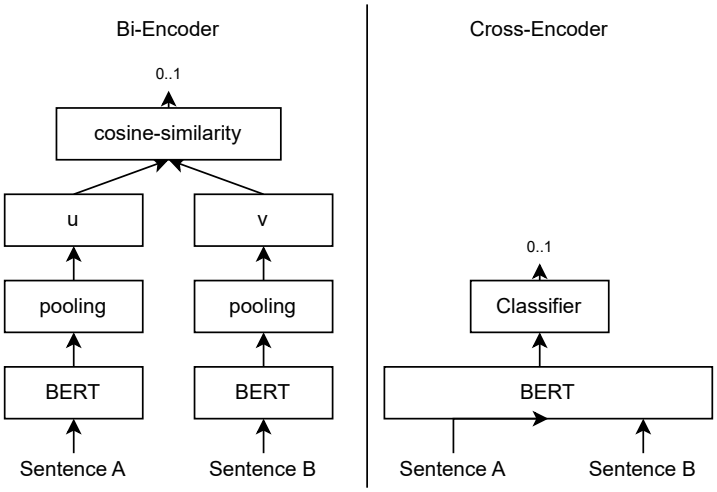


Figure 2.2: Bi vs Cross-Encoder. Taken from [17].

2.6 Normalized Discounted Cumulative Gain (NDCG)

I will evaluate the different algorithms using the Normalized Discounted Cumulative Gain (NDCG). It is a standard quality measure often used for ranking recommendation systems that score the first k results compared to an ideal result.

We first need the gains for each ranking item to calculate the cumulative gain. These can be either manually assigned or created with a more performant model. These scores can be either clamped to 0 and 1 or continuous in that interval.

To calculate the cumulative gain (CG), we take the sum of all the individual gains up to the last rank k.

$$CG = \sum_{i=1}^{k} gains$$

This does not take into account the order of the results, so we discount the gains by

rank to get the discounted cumulative gain (DCG).

$$DCG = \sum_{i=1}^{k} \frac{gains}{log_2(i+1)}$$

Lastly, we divide the discounted cumulative gain by the ideal discounted cumulative gain (IDCG), where everything is ranked in an optimal way to get the normalized discounted cumulative gain.

$$NDCG = \frac{DCG}{IDCG}$$

Since the last rank k is crucial, we usually write NDCG@k, e.g., NDCG@10.

3 Approach

This chapter will go over my solution. My solution's three major building blocks are the GUI, the recipes, and the recommendation system. It will also go over the different ranking strategies in detail.

3.1 The Jupyter Notebook GUI

For my solution, I decided to go with a Jupyter Notebook GUI. Jupyter Notebooks are already used extensively in machine learning and data science since they have many essential strengths. Being able to run code cell-by-cell, one can quickly explore and visualize data. The capability to render markdown also helps with documentation. Lastly, services like Google Colab allow for running Notebooks directly in the browser without extensive setup.

The GUI is made of three main building blocks:

1. The data grid.
2. The search box.
3. The search results.

There also exists an options dropdown menu.

3.1.1 The Data Grid and Search Box

The data grid uses the library ipydatagrid [13], a data grid widget recommended by the Jupyter Tutorial [20]. It allows for scrolling, resizing, and selecting, which is more than enough for our use case.

The search box is a Text widget.



Figure 3.1: Main parts of the recommender.

3.1.2 The Search Results

The search results are shown in a vertical list. Each result has, in order, a name, score, code snippet, copy button, and description. There are also buttons for providing feedback. The descriptions are resizable to allow for longer descriptions. The user can change the amount of returned recipes in the options.

3.1.3 Updating Results

To update the results, I used the observe methods of the various widgets to register a callback. This callback then updates the context of the recommender, recommends recipes, and changes the search results (Figure 3.2).

There is a consideration here about whether we want the callback to be triggered anytime a change is made or only after a short duration of inactivity. This could result in additional costs, especially for the OpenAI Embeddings since we send more requests than necessary. This is why the PandaRecWidget takes a debounce argument, which optionally overrides the `update_recommendation` function with a debounced version (Listing 3.1).

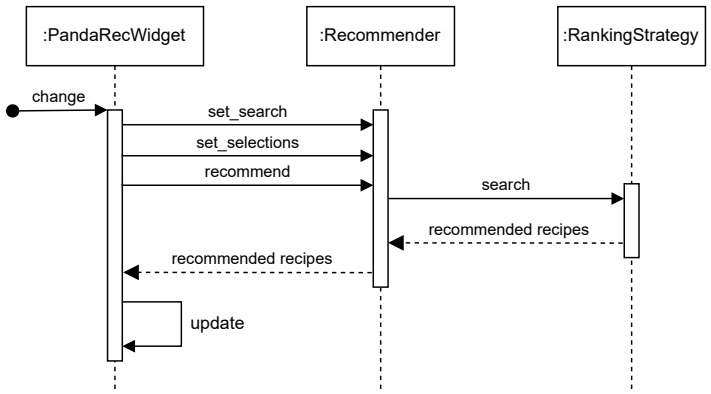


Figure 3.2: Update Results.

```python
# pandaRec/panda_rec_widget.py
if debounce:
    self.update_recommendations = debounced(0.2, self._base_update_recommendations)
else:
    self.update_recommendations = self._base_update_recommendations
```

Listing 3.1: Debounce Recommendations.

3.1.4 Applying the Code Snippet

We could apply code snippets to the underlying DataFrame in two main ways. We could use `eval()` to evaluate the code snippet directly or copy it to a new cell via user input. The `eval()` approach makes changing, e.g., row names before evaluating, more complicated and will also lose Jupyter Notebooks upside of cell-by-cell execution. By copying the snippet to the clipboard, we allow the user to choose what to do with it.

When using the Notebook online, we cannot easily access the clipboard with the Pyperclip library. That is why I created a copy button with `ipywidgets.HTML`, which calls the js clipboard api: `navigator.clipboard.writeText('{copy_text}')`.

3.2 Recipes

Recipes are a data structure consisting of a code snippet, a description, a name, and optionally keywords or argument type information.

The name and description field should give the user information about what the recipe is for. The code snippets themselves could be common functions for data manipulation,

plotting, and data import and export.

The Recipes are implemented as a Python data class since they manage data, not logic (Listing 3.2).

```python
# pandaRec/recipe.py
@dataclass
class Recipe:
    name: str
    description: str
    code: str
    keywords: str
```

Listing 3.2: Recipe Dataclass.

The recipes are bundled into a dataset and saved inside a .json file, together with the dataset description and links to other resources.

3.2.1 Generating Recipes

There are multiple ways to generate these recipes. One could:

- Generate recipes with the `dir()` method.
- Modify already existing snippets, e.g., from here: [11].
- Collect Pandas usage data from publicly available sources (Stackoverflow, CodeSearchNet) and search for often used snippets.
- Create snippets from publicly available usage guides for Pandas.

I used the first method to generate a basic set of recipes (Listing 3.3).

I also used the fourth method on the Pandas documentation, specifically "10 minutes to Pandas" and the "cookbook" from the Pandas user guide [12]. For these, I followed the VSCode snippet syntax for placeholders to make a possible later integration into VSCode easier. Lastly, I created recipes from VSCode snippets [11].

```
# generate_recipes.ipynb

# modified from https://stackoverflow.com/a/34452
def get_methods(object):
    method_list = []
    for method in dir(object):
        # 1. make sure its a function
        if not callable(getattr(object, method)):
            continue
        method_name = str(method)

        # 2. ignore internal functions
        if method_name.startswith('_'):
            continue

        # 3. only take functions with a docstring
        if str(getattr(object, method).__doc__) == 'None':
            continue
        method_list.append(
            [
                method_name,
                str(getattr(object, method).__doc__)
            ]
        )
    return method_list

recipes = [
    Recipe(method[0], method[1].strip(), "df." + method[0], "")
    for method in get_methods(df)
]
```

Listing 3.3: Generate Basic Recipes.

3.2.2 Embeddings

Some of the algorithms require some sort of pre-made representation of the data, specifically an index or an embedding. These can drastically reduce setup time and are therefore saved alongside the recipes.

3.3 The Recommendation System

3.3.1 Context

As described in Section 2.3, the context is the system's input. My system collects the user query, along with the DataFrame and selections, rectangular areas in the DataFrame that the user selected (Listing 3.4).

```python
# pandaRec/context.py
@dataclass
class Selection:
    r1: int
    c1: int
    r2: int
    c2: int


class Context:
    selections: list[Selection]
    data: DataFrame
    query: str

    def is_whole_row(self, selection: Selection) -> bool:
        """Returns True if the given Selection represents an
        entire row of the DataFrame."""
        return selection.c1 == 0 and selection.c2 == self.data.shape[1]

    def is_whole_column(self, selection: Selection) -> bool:
        """Returns True if the given Selection represents an
        entire column of the DataFrame."""
        return selection.r1 == 0 and selection.r2 == self.data.shape[0]
```

Listing 3.4: Context Class.

I also provide functions `is_whole_row()` and `is_whole_column()` so one can check whether a whole row or column is selected.

3.3.2 The Recommendation Engine

My recommendation system uses a strategy pattern for the recommendation algorithm. This makes it easy to implement multiple algorithms.

The recommender class, therefore, only holds the context, the current recommend strategy, and the list of recipes. Additionally, the strategy result is saved in the recommender as an ordered list of RecipeResults, recipes with scores (Listing 3.5).

```python
# pandaRec/recommender.py
class Recommender:
    context: Context
    strategy: RankingStrategy
    recipes: list[Recipe]
    recommended_recipes: list[RecipeResult]

    # constructor left out for brevity

    def recommend(self, num_results=10):
        """Recommends recipes with the search strategy."""
        self.recommended_recipes = self.strategy.search(
            self.context, self.recipes, num_results
        )

    def get_results(self, n=None) -> list[RecipeResult]:
        """Returns the recommended recipes.

        Args:
            n: The number of recipes to return."""

        if n is None:
            return self.recommended_recipes
        return self.recommended_recipes[:n]
```

Listing 3.5: Recommender Class.

I decided not to save a reference to the index of the corresponding recipe inside the RecipeResults because, this way, a strategy can return arbitrary recipes. Storing the

result list as a list of recipe indexes and score tuples would eliminate the data duplication, but I prefer the flexibility my approach provides. This also enables the OpenAI Chat Completion strategy (Section 3.3.3).

3.3.3 Ranking Strategies

For my solution, I implemented multiple ranking strategies to compare them. These can be swapped out easily because of the strategy pattern (Listing 3.6).

```
# pandaRec/strategies.py
from abc import ABC, abstractmethod

class RankingStrategy(ABC):
@abstractmethod
    def search(self, context: Context, recipes: list[Recipe], num_results=10)
    -> list[RecipeResult]:
        """Searches for recipes based on the context."""

    def feedback(self, context: Context, recipe: Recipe, positive: bool):
        """Provides feedback to the ranking strategy."""

```

Listing 3.6: RankingStrategy Abstract Class.

Any strategy we add will inherit this abstract class and provide its search function. The recommender will call this function as shown in Listing 3.5. All my strategies score the recipes in the interval $[0, 1]$. Let us first look at a basic strategy to understand how this works.

Name Search

Name search searches for the query string in the name of the recipes, scoring it one if it finds it and not returning the recipe at all if the query string is absent. This ends up in name search working like code completion since the names of the auto-generated recipes are the same as the function names (Listing 3.7).

```python
# pandaRec/strategies.py
class NameSearch(RankingStrategy):
    """A simple proof-of-concept ranking strategy that only
    searches for the query in the recipe name."""

    @staticmethod
    def search(
        context: Context, recipes: list[Recipe], num_results=10
    ) -> list[RecipeResult]:
        result = []
        for recipe in recipes:
            if context.query in recipe.name:
                result.append(RecipeResult(1, recipe))
    return result[:num_results]
```

Listing 3.7: Name Search.

We can see with this example how a ranking algorithm will work; it will iterate over the recipes, score them, sort them (not the case here since the scores are all 1), and then return them to the recommender.

Fuzzy Search

To implement Fuzzy Search I use the library "rapidfuzz" to calculate the lexical distances between the search query and the descriptions. Their `partial_token_sort_ratio` function sorts the words and calculates the partial ratio between the prompt and the description (Listing 3.8).

I use a library for this because performance starts to matter here. The partial ratio they use is implemented in C++ and runs for short queries ($n<64$) in worst-case $O(nm)$ time, with m being the number of different alignments of the shorter string in the bigger string, so effectively the size of the bigger string if they are large enough. Since we must iterate over all recipes, we get $O(nmr)$, which can incur noticeable slowdowns (in the

```python
# pandaRec/strategies.py FuzzySearchDescription class
def __init__(self, ratio=fuzz.partial_token_sort_ratio):
    """Initializes the fuzzy search ranking with a given rapidfuzz ratio."""
    super().__init__()
    self.ratio = ratio

def search(
    self, context: Context, recipes: list[Recipe], num_results=10
) -> list[RecipeResult]:
    result = []
    for recipe in recipes:
        score = self.ratio(context.query, recipe.description,
                  processor=default_process)
        result.append(RecipeResult(score, recipe))
    result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
    return result[:num_results]
```

Listing 3.8: Fuzzy Search.

decisecond range) on web-hosted Jupyter engines. So, let us look at an algorithm that still allows us to find partial matches and matches of similar words while simultaneously being way faster.

Index Search

I have already introduced index search in Section 2.5.2. When we construct the strategy, we build the index only if we do not provide a path to a prebuilt index (Listing 3.9).

This path can be saved alongside the list of recipes in the .json, as mentioned in Section 3.2. Before generating the index, we first want to lemmatize the descriptions (see Section 2.5.1). For this, we start by tokenizing the descriptions with the regex `"\w*[a-zA-Z]\w*"`, which matches words with at least one letter. We then use the Natural Language Toolkit NLTK to lemmatize the words and remove common stop words like "a" or "the" that are irrelevant to the search (Listing 3.10).

We can then build the forward index and, lastly, the inverted index (Listing 3.11).

```python
# pandaRec/strategies.py IndexSearch class
def __init__(self, recipes: list[Recipe], path: str = ""):
    """The search index can be generated on the fly or loaded from a file."""
    super().__init__()
    if not path:
        self.index = generate_search_index(
            [recipe.description for recipe in recipes]
        )
    else:
        self.index = load_search_index(path)
```

Listing 3.9: Index Search Constructor.

```python
# pandaRec/search_index.py
def lemmatize_no_stop_words(text: str) -> list[str]:
    """Lemmatizes the given text and removes stop words.
    Stop words are words that are very common and thus do not carry much meaning."""
    # download the NLTK wordnet and related resources
    nltk.download("wordnet", quiet=True)
    nltk.download("omw-1.4", quiet=True)
    nltk.download("stopwords", quiet=True)
    words = re.findall(r"\w*[a-zA-Z]\w*", text)
    wnl = WordNetLemmatizer()
    return [
        wnl.lemmatize(word) for word in words
        if word not in stopwords.words("english")
    ]
```

Listing 3.10: Lemmatization.

```python
# pandaRec/search_index.py
def generate_search_index(descriptions: list[str])
        -> dict[str, list[tuple[int, int]]]:
    """Generates a search index from the given list of descriptions."""
    # 1. Lemmatize the descriptions
    lemmatized_descriptions = [
            lemmatize_no_stop_words(description) for description in descriptions
        ]
    # 2. build the forward index, one list of (word, frequenzy) tuples
    # per description
    frequency_per_description = [
            [(word, description.count(word)) for word in set(description)]
            for description in lemmatized_descriptions
        ]
        # 3. construct the inverted index, one list of
        # (description_index, frequency) tuples per word
    frequency_per_word = {}
    for idx, description in enumerate(frequency_per_description):
        for (word, count) in description:
            if word not in frequency_per_word:
                frequency_per_word[word] = [(idx, count)]
            else:
                frequency_per_word[word].append((idx, count))
    return frequency_per_word
```

Listing 3.11: Building the Inverted Index.

We can then pickle the hashmap to save it to disk. When searching for a query, we lemmatize it and normalize how many query words are in each document (Listing 3.12).

We also must sort the list of results since we want this list to be in the order of ranking.

Semantic Search

The basics of semantic search were already described in Section 2.5.3. To implement a semantic search, we will use the Sentence-Transformers library, which is originally based on [8]. Like constructing the reverse index in the index-based search, we first must create embeddings for all our recipes. This is again handled in the constructor (Listing 3.13).

```python
# pandaRec/strategies.py IndexSearch class
def search(
    self, context: Context, recipes: list[Recipe], num_results=10
) -> list[RecipeResult]:
    lemmatized_search = lemmatize_no_stop_words(context.query)

    # add 1 to the score for each word that is in the description (per recipe)
    scores = len(recipes) * [0]
    for word in lemmatized_search:
        if word in self.index:
            for idx, _ in self.index[word]:
                scores[idx] += 1

    num_words = len(lemmatized_search)
    result = [
            RecipeResult(score / num_words, recipe) # normalize score
            # scores and recipes are in the same order so we can zip() them
            for score, recipe in zip(scores, recipes)
        ]
    result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
    return result[:num_results]
```

Listing 3.12: Searching the Inverted Index.

```python
# pandaRec/strategies.py SemanticSearch class
def __init__(self, recipes: list[Recipe], path: str = "",
    model: str = "sentence-transformers/all-mpnet-base-v2"):
    """The embeddings can be generated on the fly or loaded from a file."""
    super().__init__()
    self.model = SentenceTransformer(model)
    descriptions = [recipe.description for recipe in recipes]
    if not path:
        self.embeddings = self.model.encode(descriptions)
    else:
        self.embeddings = load_embeddings(path)
```

Listing 3.13: Embedding the Recipes.

We can then embed our query and use the cosine similarity to search for descriptions with similar semantic meanings (Listing 3.14). I have tested this algorithm with three different models: `all-mpnet-base-v2` and `all-MiniLM-L6-v2` which are general general-purpose, and `krlvi/sentence-t5-base-nlpl-code_search_net` which is trained with the CodeSearchNet [6] dataset. In Section 4.4.3 I will evaluate the differences between these models.

```
# pandaRec/strategies.py SemanticSearch class
def search(self, context: Context, recipes: list[Recipe], num_results=10)
    -> list[RecipeResult]:
    query_embedding = self.model.encode(context.query, convert_to_tensor=True)
    cos_scores = [
        util.cos_sim(query_embedding, embedding).item(
        for embedding in self.embeddings
    ]
    result = [
        RecipeResult(score, recipe)
        for recipe, score in zip(recipes, cos_scores)
    ]
    result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
    return result[:num_results]
```

Listing 3.14: Searching the Embeddings.

OpenAI Embeddings

We can also use OpenAIs API to generate embeddings (Listing 3.15). Their function `get_embeddings` automatically manages batching, but the process is other than that the same as with Sentence-Transformers. The ranking process is the same as in Listing 3.14.

OpenAI Chat Completion

Instead of using OpenAI's API to generate embeddings, we can use the chat completion generative AI to freely generate recipes. For this I use this system message (Listing 3.16):

This takes advantage of the fact that RecipeResults are not bound to the predefined recipes.

```python
# pandaRec/strategies.py OpenAIEmbeddings class
def __init__(self, recipes: list[Recipe], path: str = "",
             model: str = "text-embedding-ada-002"):
    """The embeddings can be generated on the fly or loaded from a file."""
    super().__init__()
    self.model = model
    descriptions = [recipe.description for recipe in recipes]
    if not path:
        self.embeddings = get_embeddings(descriptions, engine=self.model)
    else:
        self.embeddings = load_embeddings(path)
```

Listing 3.15: OpenAI embeddings.

```
You are a code recommendation system that recommends code snippets from
the python pandas library that can manipulate a DataFrame based on a user query.
The recommendations should have a name, a code snippet and a description.
The format is:
name: <name>
code: <code>
description: <description>
You will return at most 5 code snippets that are relevant to the task
in order of relevance.
```

Listing 3.16: OpenAI System Message.

Web-Based Search

Using WebSockets, we can offload the ranking algorithm to a server. This server can process and deliver recommendations to multiple clients simultaneously. But, more importantly, we can collect usage data from the clients to improve our recipe dataset and the Semantic Search model.

In my implementation, I send the query and the wanted number of results as a JSON string and receive that many RecipeResults. There is a small detail here where Jupyter Notebooks already uses an async loop, and I therefore use the package `nest_asyncio` so i can still use `asyncio.run()` (Listing 3.17).

```python
# pandaRec/strategies.py WebSocketStrategy class
async def asearch(
    self, context: Context, _recipes: list[Recipe], num_results=10)
-> str:
    """async search"""
    payload = json.dumps({
        "type": "search",
        "query": context.query,
        "num_results": num_results
    })
    async with websockets.connect(self.uri) as websocket:
        await websocket.send(payload)
        result = await websocket.recv()
    return result

def search(self, context: Context, recipes: list[Recipe], num_results):
    text_result = asyncio.run(self.asearch(context, recipes, num_results))
    result = self.recipe_results_from_json(text_result)
    return result
```

Listing 3.17: Web Socket Client.

3.4 Collecting User Feedback

For now, we only looked at the search method of the RankingStrategy abstract class. There is a second method that is for collecting user feedback. Pressing one of the feedback buttons allows the user to provide positive or negative feedback for a recommendation. We can collect this feedback on the server and use it to fine-tune our models in the future.

4 Evaluation

This chapter evaluates the different ranking algorithms concerning their setup speed, search speed, and ranking performance. It will also compare the different recipe datasets.

4.1 Research Questions

Our solution should be fast, easy to set up, use, and useful to all users. I will start by testing the speed of the system. For this, I will decide what an acceptable delay is and then test if the algorithms are faster than this acceptable delay. To evaluate the usefulness of my solution, I will calculate the accuracy of the algorithms and compare them to ChatGPT.

4.2 Acceptable Delays

I consider everything with a delay under 50 ms real-time. Over 100 ms, the search could take longer than the delay between typed characters. Jupyter Notebooks automatically only handle input after all Python code has finished running. Adding a debounce (Listing 3.1) is recommended if the recommender takes longer than the delay between two character inputs. A delay over a few seconds is no longer acceptable.

4.3 Setup and Search Delays

All delays were timed on an AMD Ryzen 7 5800X3D CPU. To evaluate delays in the system, we can use Jupyter Notebooks inbuild `%timeit` magic command, which uses the timeit module to time the execution of a statement. For example, this is what timing the setup time for index search with a saved index looks like (Listing 4.1).

If we do this for all implemented algorithms, we can compare their setup and search time. I have used the query "fill missing values with zero" to analyze search time.

```
%timeit IndexSearch(recipes, '../recipes/search_index.pkl')

# 1.50 ms ± 3.07 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Listing 4.1: Timing index search with a saved index.

Name	Setup Delay	Search Delay
Name Search	68.0 ns ± 0.2 ns	6.53 µs ± 0.08 µs
Fuzzy Name Search	68.4 ns ± 0.5 ns	756.0 µs ± 1.3 µs
Fuzzy Description Search	276.0 ns ± 0.8 ns	10.00 ms ± 0.05 ms
Index Search	7.87 s ± 0.08 s	1.74 ms ± 0.01 ms
Semantic Search	85 s ± 1 s	38.2 ms ± 0.9 ms
OpenAI Embedding	3 s ± 1 s	668 ms ± 949 ms
OpenAI Chat Completion	530.0 ns ± 14.9 ns	27.4 s ± 3.7 s
Saved Index/ Embedding		
Index Search	1.50 ms ± 0.03 ms	-
Semantic Search	682 ms ± 10 ms	-
OpenAI Embedding	27.50 ms ± 0.25 ms	-

Table 4.1: Setup and search delay for all implemented algorithms.

Name search and fuzzy name search both do not have any setup, and fuzzy description search only registers the fuzzy ratio algorithm, which results in basically instant setup time. Index search takes about 8 seconds to build the index. The embedding using Sentence Transformers takes about one and a half minutes to embed all the descriptions.

This could be sped up by using the GPU by passing `device="cuda"` to the SentenceTransformer constructor. We can eliminate most of the setup time by using a precalculated index or embedding.

The search delay of most algorithms is under 50 ms, but OpenAI Embeddings take noticeably longer and need a debounce. OpenAI Chat Completion takes the longest, well over what I consider acceptable for this setup.

4.3.1 Setup Time of the Solution

My solution compares multiple different algorithms, resulting in many needed packages. This, in turn, increases the time needed to set up an environment in, e.g., Google Colab. I recommend getting rid of all algorithms except Semantic Search for a usable version.

4.4 Algorithm Accuracy

To evaluate the accuracy of the algorithms, we will use the NDCG introduced in Section 2.6.

4.4.1 Creating the Ranking Dataset

There are no good pre-existing datasets for ranking the algorithms, so I will create my own. To do this, I selected the article [21]. For every one of the thirty functions, I created a description and selected recipes that can fulfill these goals, which I assigned the score one. I then calculated the NDCG@5 for each of the algorithms. For some of the descriptions, none of the recipes can fulfill the goal. These are not counted towards the NDCG@5 score (since the IDCG@5 is 0, breaking our formula).

4.4.2 Ranking Based on the Autogenerated Recipes

Since none of the ranking descriptions are substrings of the recipes' names, Name Search does not recommend anything and, therefore, always scores zero. This is fine since it worked more as a proof of concept.

Name	Score
Name Search	0
Fuzzy Name Search	0.39
Fuzzy Description Search	0.26
Index Search	0.21
Semantic Search	0.62
OpenAI Embedding	0.54

Table 4.2: NDCG Scores for all implemented algorithms.

The Lexical search algorithms work noticeably worse. This is probably because of the shortness of our descriptions and the technical language used in the descriptions.

4.4.3 Comparing Semantic Search Models

Now, let us compare the different models I considered for the semantic search using Sentence-Transformers.

all-mpnet-base-v2 is the general-purpose model from Sentence-Transformers with the best quality. It was partly trained on CodeSearchNet.

all-MiniLM-L6-v2 is a smaller general-purpose model that the Sentence-Transformer documentation claims is five times faster at the cost of some performance. It was also partly trained on CodeSearchNet.

sentence-t5-base is the base model of the t5 architecture, which is trained on predicting multiple masked words simultaneously.

krlvi/sentence-t5-base-nlpl-code_search_net is based on a t5-base checkpoint and then trained on CodeSearchNet.

Name	Score	Setup Delay	Search Delay
all-mpnet-base-v2	0.62	85 s ± 1 s	38.2 ms ± 0.9 ms
all-MiniLM-L6-v2	0.50	10.3 s ± 0.1 s	16.4 ms ± 0.6 ms
sentence-t5-base	0.53	52.3 s ± 0.5 s	40.6 ms ± 9.2 ms
sentence-t5-base-...	0.48	50.8 s ± 0.5 s	39.6 ms ± 1.1 ms

Table 4.3: NDCG Scores for different models.

Using all-MiniLM-L6-v2 resulted in an 8.3× speedup in setup delay and a 2.3× speedup in searching delay. The model trained on CodeSearchNet performs worse. This is because CodeSearchNet provides a text-to-code dataset, and we are evaluating a text-to-text task.

4.4.4 Performance of the Different Recipe Datasets

We can use the same idea to rank the different recipe datasets: the one created from docstrings, the one created from the Pandas cookbook, and the one created from VSCode snippets.

	docstring	cookbook	snippets
NameSearch	0.00	0.00	0.00
FuzzySearchName	0.41	0.45	0.53
FuzzySearchDescription	0.27	0.31	0.48
IndexSearch	0.22	0.40	0.29
SemanticSearch	0.64	0.71	0.80
OpenAIEmbeddings	0.57	0.44	0.88

Table 4.4: NDCG Scores for different datasets.

The recipes generated with VSCode snippets outperform the other two datasets. However, this table hides how many queries cannot be achieved by any dataset. Let us look at the number of unachievable recipes per dataset and the average number of achievable recipes per successful query.

	docstring	cookbook	snippets
Amount of recipes	178	29	54
Nonachievable queries	6	21	16
Average amount	1.09	1.13	1.77

Table 4.5: Unachievable Recipes per Dataset.

The datasets with more recipes in total can also cover more queries. The snippets dataset has a higher concentration since it focuses only on common tasks. The docstring dataset is better in a broader set of circumstances, and the snippets dataset is more effective when the user uses common techniques.

4.4.5 Comparison to OpenAI Chat Completion

OpenAI Chat Completion does not use the predefined recipes, meaning that the NDCG cannot be used to rank it. When I searched for all the queries in the test set, it generally only returned correct answers. However, some of the answers did not follow my provided structure and could not be coerced into a RecipeResult. This occurrence could be reduced by writing a better system message.

4.4.6 Comparison to ChatGPT

I used the query: "I am searching for pandas functions that can do the following:" followed by the queries with ChatGPT and mainly got correct answers. The query asking for a Pandas function that can "change a list of data into unique entries" was, however, wrongly interpreted as `df['column_name'].unique()` instead of `df.explode(column)`. The whole discussion can be found on [22].

4.4.7 Usefulness of Recipes

Since both the OpenAI Chat Completion and ChatGPT perform decently on this task, there is the question of whether using my solution is an improvement. I will compare these systems to the semantic search algorithm.

When it comes to search delays, semantic search is faster. Accuracy is better with both the Chat Completion and ChatGPT, but the accuracy of semantic search can still be improved by improving the recipes and fine-tuning the semantic search model. Regarding ease of use, my all-in-one solution is more straightforward than using an IDE and a browser for ChatGPT, but the setup time of my solution does decrease its usefulness. Getting rid of all algorithms except semantic search or the web-based algorithm should decrease the setup time substantially and keep it competitive.

ChatGPT can also provide help with setting up the environment, debugging errors, and correcting itself, which is impossible with semantic search. Competing with such a heavy hitter like OpenAI is not easy, but I do think my approach has merit.

4.5 Working With Jupyter Notebooks

I decided to implement my GUI in Jupyter Notebooks. They are easy to set up and effective at interacting with data. Additionally, the closeness between code and UI should help with learning the Pandas library faster. They also allow for easy sharing of my solution. However, there were also some problems with them.

Different Websites use different versions of standard Python libraries, which can result in the Notebook behaving differently. For example, at the time of writing, there is an open bug where ipydatagrid does not render on Google Colab [15]. Downgrading to version 1.1.16 fixes it, but it is definitely confusing.

Also, Notebooks does not have functionality that full-fledged GUIs offer. When we change the DataFrame with a copied function or, in general, we would like to update the recommender's DataFrame and the data grid automatically. However, this is currently not supported by the widgets library [19].

5 Conclusion

This chapter concludes the thesis with a summary and possible future work.

5.1 Summary

Of all the tested algorithms, semantic search provides great results regarding accuracy and speed. Lexical search did not provide good accuracy but was still fast.

Using Jupyter for the GUI was great for prototyping, fast iteration, and its ability to run code cell-by-cell.

Using recipes together with semantic search is fast, easy to set up, use, and useful.

While my approach is not more accurate than current state-of-the-art systems like ChatGPT, it is faster and easier to use. ChatGPT's usefulness for its versatility still means it outperforms my solution, but since the ranking model can still be improved by fine-tuning and the recipes can be further enhanced, I think my approach has merit.

5.2 Future Work

This work provides multiple approaches for future work.

Improving the Ranking Strategy The ranking strategy can be improved by fine-tuning or by using Retrieve and Re-Rank [18], which uses a bi-encoder to retrieve a smaller list of recipes and then a cross-encoder to get more accurate scores on this smaller list.

Improving the Recipes The recipes can see improvements in the quality of their descriptions and the number of recipes.

UI My solution can be ported into a standalone UI or as a plugin for IDEs such as VSCode. The modularity of my system should make this trivial.

Other Algorithms The structure of my solution can be used as a springboard to implement and compare new algorithms.

Bibliography

[1] Martin Robillard, Robert Walker, and Thomas Zimmermann. "Recommendation Systems for Software Engineering". In: *IEEE Software* 27.4 (2010), pp. 80–86. DOI: 10.1109/MS.2009.161.

[2] Abhishek Das and Ankit Jain. "Indexing the world wide web: The journey so far". In: *Next Generation Search Engines: Advanced Models for Information Retrieval*. IGI Global, 2012, pp. 1–28.

[3] Hannah Bast, Björn Buchhold, Elmar Haussmann, et al. "Semantic search on text and knowledge bases". In: *Foundations and Trends® in Information Retrieval* 10.2-3 (2016), pp. 119–271.

[4] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[5] Alec Radford et al. *Improving language understanding with unsupervised learning*. Tech. rep. OpenAI, 2018.

[6] Hamel Husain et al. "Codesearchnet challenge: Evaluating the state of semantic code search". In: *arXiv preprint arXiv:1909.09436* (2019).

[7] Yinhan Liu et al. "Roberta: A robustly optimized bert pretraining approach". In: *arXiv preprint arXiv:1907.11692* (2019).

[8] Nils Reimers and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks". In: *arXiv preprint arXiv:1908.10084* (2019).

[9] Zhangyin Feng et al. "Codebert: A pre-trained model for programming and natural languages". In: *arXiv preprint arXiv:2002.08155* (2020).

[10] Karpathy Andrej et al. *Generative models*. Sept. 2023. URL: https://openai.com/research/generative-models.

[11] Raza Balbale. *Basic Pandas Snippets*. Sept. 2023. URL: https://github.com/snippington/snp-pandas-basic/blob/main/snippets/pandas-basic.code-snippets.

[12] Pandas Community. *Pandas Documentation*. Sept. 2023. URL: https://pandas.pydata.org/docs/.

[13] Bloomberg Engineering. *ipydatagrid*. Sept. 2023. URL: https://github.com/bloomberg/ipydatagrid.

[14] GitHub and OpenAI. *GitHub Copilot*. Sept. 2023. URL: https://github.com/features/copilot.

[15] lys1900. *Bug with colab and ipydatagrid*. Sept. 2023. URL: https://github.com/bloomberg/ipydatagrid/issues/444.

[16] OpenAI. *ChatGPT Website*. Sept. 2023. URL: https://chat.openai.com/.

[17] Nils Reimers. *Sentence-Transformers Documentation*. Sept. 2023. URL: https://www.sbert.net/examples/applications/cross-encoder/README.html.

[18] Nils Reimers. *Sentence-Transformers Documentation*. Sept. 2023. URL: https://www.sbert.net/examples/applications/retrieve_rerank/README.html.

[19] Martin Renou and Wolf Vollprecht. *Widgets does not support observing DataFrames*. Sept. 2023. URL: https://github.com/bloomberg/ipydatagrid/issues/6.

[20] Veit Schiele. *Jupyter Tutorial*. Sept. 2023. URL: https://jupyter-tutorial.readthedocs.io/en/v1.1.0/ipywidgets/libs/index.html.

[21] Rashida Sucky. *30 Very Useful Pandas Functions for Everyday Data Analysis Tasks*. Sept. 2023. URL: https://regenerativetoday.com/30-very-useful-pandas-functions-for-everyday-data-analysis-tasks/.

[22] Frederick Vandermoeten and ChatGPT. *ChatGPT Chat*. Oct. 2023. URL: https://chat.openai.com/share/12f5dd08-59ff-461e-9aba-c3b485b8c632.